



```
In [11]: import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
In [2]: def load_boston_housing_data():

    # Load CSV file (make sure file is in same folder)
    df = pd.read_csv("HousingData.csv")

    # Handle missing values (important for this dataset)
    df = df.fillna(df.mean())

    # Split features and target
    X = df.iloc[:, :-1].values    # all columns except last
    y = df.iloc[:, -1].values    # last column (MEDV)

    # Normalize data
    scaler_X = StandardScaler()
    scaler_y = StandardScaler()

    X = scaler_X.fit_transform(X)
    y = scaler_y.fit_transform(y.reshape(-1, 1)).flatten()

    return X, y, scaler_y

# Load data
X, y, scaler_y = load_boston_housing_data()
```

```
In [3]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42
)

# Convert to float32 (important)
X_train = X_train.astype('float32')
X_val = X_val.astype('float32')
X_test = X_test.astype('float32')

y_train = y_train.astype('float32')
y_val = y_val.astype('float32')
y_test = y_test.astype('float32')
```

```
In [4]: batch_size = 32

train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train)) \
```

```

        .shuffle(buffer_size=len(X_train)) \
        .batch(batch_size) \
        .prefetch(tf.data.AUTOTUNE)

val_dataset = tf.data.Dataset.from_tensor_slices((X_val, y_val)) \
    .batch(batch_size) \
    .prefetch(tf.data.AUTOTUNE)

test_dataset = tf.data.Dataset.from_tensor_slices((X_test, y_test)) \
    .batch(batch_size) \
    .prefetch(tf.data.AUTOTUNE)

```

```

In [5]: model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)), # dynamic input

    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.BatchNormalization(),

    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.BatchNormalization(),

    tf.keras.layers.Dense(16, activation='relu'),

    tf.keras.layers.Dense(1) # Linear output (regression)
])

# Compile model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

```

In [6]: callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=20,
        restore_best_weights=True,
        verbose=1
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=10,
        min_lr=0.00001,
        verbose=1
    )
]

```

```
In [7]: history = model.fit(  
        train_dataset,  
        epochs=200,  
        validation_data=val_dataset,  
        callbacks=callbacks,  
        verbose=1  
    )
```

Epoch 1/200
11/11  7s 72ms/step - loss: 1.7520 - mae: 0.9305 - val_loss: 0.7320 - val_mae: 0.6097 - learning_rate: 0.0010
Epoch 2/200
11/11  1s 14ms/step - loss: 0.8220 - mae: 0.6493 - val_loss: 0.5968 - val_mae: 0.5464 - learning_rate: 0.0010
Epoch 3/200
11/11  0s 14ms/step - loss: 0.4610 - mae: 0.4966 - val_loss: 0.5459 - val_mae: 0.5294 - learning_rate: 0.0010
Epoch 4/200
11/11  0s 12ms/step - loss: 0.3596 - mae: 0.4409 - val_loss: 0.5175 - val_mae: 0.5281 - learning_rate: 0.0010
Epoch 5/200
11/11  0s 12ms/step - loss: 0.3128 - mae: 0.4171 - val_loss: 0.4835 - val_mae: 0.5132 - learning_rate: 0.0010
Epoch 6/200
11/11  0s 13ms/step - loss: 0.2731 - mae: 0.3821 - val_loss: 0.4626 - val_mae: 0.5000 - learning_rate: 0.0010
Epoch 7/200
11/11  0s 14ms/step - loss: 0.2492 - mae: 0.3794 - val_loss: 0.4510 - val_mae: 0.4931 - learning_rate: 0.0010
Epoch 8/200
11/11  0s 13ms/step - loss: 0.2802 - mae: 0.4044 - val_loss: 0.4349 - val_mae: 0.4823 - learning_rate: 0.0010
Epoch 9/200
11/11  0s 12ms/step - loss: 0.2500 - mae: 0.3824 - val_loss: 0.4335 - val_mae: 0.4870 - learning_rate: 0.0010
Epoch 10/200
11/11  0s 12ms/step - loss: 0.2375 - mae: 0.3720 - val_loss: 0.4186 - val_mae: 0.4723 - learning_rate: 0.0010
Epoch 11/200
11/11  0s 12ms/step - loss: 0.1809 - mae: 0.3203 - val_loss: 0.4149 - val_mae: 0.4703 - learning_rate: 0.0010
Epoch 12/200
11/11  0s 13ms/step - loss: 0.2346 - mae: 0.3561 - val_loss: 0.4015 - val_mae: 0.4580 - learning_rate: 0.0010
Epoch 13/200
11/11  0s 12ms/step - loss: 0.1838 - mae: 0.3247 - val_loss: 0.3886 - val_mae: 0.4546 - learning_rate: 0.0010
Epoch 14/200
11/11  0s 13ms/step - loss: 0.1815 - mae: 0.3279 - val_loss: 0.3829 - val_mae: 0.4484 - learning_rate: 0.0010
Epoch 15/200
11/11  0s 12ms/step - loss: 0.2143 - mae: 0.3369 - val_loss: 0.3800 - val_mae: 0.4392 - learning_rate: 0.0010
Epoch 16/200
11/11  0s 12ms/step - loss: 0.1576 - mae: 0.3075 - val_loss: 0.3746 - val_mae: 0.4322 - learning_rate: 0.0010
Epoch 17/200
11/11  0s 12ms/step - loss: 0.1760 - mae: 0.3287 - val_loss: 0.3676 - val_mae: 0.4230 - learning_rate: 0.0010
Epoch 18/200
11/11  0s 12ms/step - loss: 0.1638 - mae: 0.3137 - val_loss: 0.3654 - val_mae: 0.4167 - learning_rate: 0.0010

Epoch 19/200
11/11  0s 13ms/step - loss: 0.1655 - mae: 0.3138 - val_loss: 0.3388 - val_mae: 0.3945 - learning_rate: 0.0010
Epoch 20/200
11/11  0s 12ms/step - loss: 0.1759 - mae: 0.3186 - val_loss: 0.3400 - val_mae: 0.3994 - learning_rate: 0.0010
Epoch 21/200
11/11  0s 12ms/step - loss: 0.1612 - mae: 0.3038 - val_loss: 0.3385 - val_mae: 0.3918 - learning_rate: 0.0010
Epoch 22/200
11/11  0s 12ms/step - loss: 0.1230 - mae: 0.2606 - val_loss: 0.3272 - val_mae: 0.3856 - learning_rate: 0.0010
Epoch 23/200
11/11  0s 12ms/step - loss: 0.1492 - mae: 0.2960 - val_loss: 0.3235 - val_mae: 0.3827 - learning_rate: 0.0010
Epoch 24/200
11/11  0s 13ms/step - loss: 0.1197 - mae: 0.2709 - val_loss: 0.3224 - val_mae: 0.3859 - learning_rate: 0.0010
Epoch 25/200
11/11  0s 12ms/step - loss: 0.1729 - mae: 0.2962 - val_loss: 0.3308 - val_mae: 0.3923 - learning_rate: 0.0010
Epoch 26/200
11/11  0s 12ms/step - loss: 0.1609 - mae: 0.2938 - val_loss: 0.3167 - val_mae: 0.3770 - learning_rate: 0.0010
Epoch 27/200
11/11  0s 12ms/step - loss: 0.1403 - mae: 0.2898 - val_loss: 0.3197 - val_mae: 0.3753 - learning_rate: 0.0010
Epoch 28/200
11/11  0s 12ms/step - loss: 0.1349 - mae: 0.2837 - val_loss: 0.3137 - val_mae: 0.3700 - learning_rate: 0.0010
Epoch 29/200
11/11  0s 12ms/step - loss: 0.1323 - mae: 0.2774 - val_loss: 0.3136 - val_mae: 0.3750 - learning_rate: 0.0010
Epoch 30/200
11/11  0s 12ms/step - loss: 0.1610 - mae: 0.2960 - val_loss: 0.3085 - val_mae: 0.3728 - learning_rate: 0.0010
Epoch 31/200
11/11  0s 11ms/step - loss: 0.1348 - mae: 0.2719 - val_loss: 0.3207 - val_mae: 0.3811 - learning_rate: 0.0010
Epoch 32/200
11/11  0s 11ms/step - loss: 0.1326 - mae: 0.2752 - val_loss: 0.3410 - val_mae: 0.3912 - learning_rate: 0.0010
Epoch 33/200
11/11  0s 11ms/step - loss: 0.1330 - mae: 0.2755 - val_loss: 0.3364 - val_mae: 0.3839 - learning_rate: 0.0010
Epoch 34/200
11/11  0s 12ms/step - loss: 0.1661 - mae: 0.3135 - val_loss: 0.3277 - val_mae: 0.3756 - learning_rate: 0.0010
Epoch 35/200
11/11  0s 10ms/step - loss: 0.1249 - mae: 0.2675 - val_loss: 0.3298 - val_mae: 0.3764 - learning_rate: 0.0010
Epoch 36/200
11/11  0s 10ms/step - loss: 0.1280 - mae: 0.2807 - val_loss: 0.3390 - val_mae: 0.3843 - learning_rate: 0.0010

Epoch 37/200
11/11  **0s** 11ms/step - loss: 0.1290 - mae: 0.2708 - val_loss: 0.3389 - val_mae: 0.3813 - learning_rate: 0.0010
Epoch 38/200
11/11  **0s** 21ms/step - loss: 0.1363 - mae: 0.2789 - val_loss: 0.3305 - val_mae: 0.3748 - learning_rate: 0.0010
Epoch 39/200
11/11  **0s** 11ms/step - loss: 0.1154 - mae: 0.2643 - val_loss: 0.3308 - val_mae: 0.3800 - learning_rate: 0.0010
Epoch 40/200
9/11  **0s** 6ms/step - loss: 0.1634 - mae: 0.2938
Epoch 40: ReduceLRonPlateau reducing learning rate to 0.00020000000949949026.
11/11  **0s** 12ms/step - loss: 0.1396 - mae: 0.2820 - val_loss: 0.3291 - val_mae: 0.3748 - learning_rate: 0.0010
Epoch 41/200
11/11  **0s** 10ms/step - loss: 0.1153 - mae: 0.2657 - val_loss: 0.3219 - val_mae: 0.3694 - learning_rate: 2.0000e-04
Epoch 42/200
11/11  **0s** 11ms/step - loss: 0.1380 - mae: 0.2729 - val_loss: 0.3217 - val_mae: 0.3688 - learning_rate: 2.0000e-04
Epoch 43/200
11/11  **0s** 11ms/step - loss: 0.1356 - mae: 0.2773 - val_loss: 0.3184 - val_mae: 0.3670 - learning_rate: 2.0000e-04
Epoch 44/200
11/11  **0s** 11ms/step - loss: 0.1119 - mae: 0.2563 - val_loss: 0.3170 - val_mae: 0.3670 - learning_rate: 2.0000e-04
Epoch 45/200
11/11  **0s** 10ms/step - loss: 0.1166 - mae: 0.2638 - val_loss: 0.3169 - val_mae: 0.3684 - learning_rate: 2.0000e-04
Epoch 46/200
11/11  **0s** 10ms/step - loss: 0.1162 - mae: 0.2577 - val_loss: 0.3207 - val_mae: 0.3710 - learning_rate: 2.0000e-04
Epoch 47/200
11/11  **0s** 11ms/step - loss: 0.1068 - mae: 0.2480 - val_loss: 0.3234 - val_mae: 0.3730 - learning_rate: 2.0000e-04
Epoch 48/200
11/11  **0s** 11ms/step - loss: 0.1087 - mae: 0.2501 - val_loss: 0.3207 - val_mae: 0.3708 - learning_rate: 2.0000e-04
Epoch 49/200
11/11  **0s** 11ms/step - loss: 0.1196 - mae: 0.2646 - val_loss: 0.3190 - val_mae: 0.3695 - learning_rate: 2.0000e-04
Epoch 50/200
10/11  **0s** 6ms/step - loss: 0.0941 - mae: 0.2336
Epoch 50: ReduceLRonPlateau reducing learning rate to 4.0000001899898055e-05.
11/11  **0s** 10ms/step - loss: 0.1026 - mae: 0.2459 - val_loss: 0.3174 - val_mae: 0.3683 - learning_rate: 2.0000e-04
Epoch 50: early stopping
Restoring model weights from the end of the best epoch: 30.

```
In [8]: test_loss, test_mae = model.evaluate(test_dataset)

print("\nTest Loss (MSE):", test_loss)
print("Test MAE:", test_mae)
```

4/4 ————— 0s 8ms/step - loss: 0.2431 - mae: 0.3442

Test Loss (MSE): 0.24314959347248077

Test MAE: 0.3442215621471405

```
In [9]: predictions = model.predict(test_dataset)

# Convert back to original scale
predictions_rescaled = scaler_y.inverse_transform(predictions)
y_test_rescaled = scaler_y.inverse_transform(y_test.reshape(-1, 1))

print("\nSample Predictions:")
for i in range(5):
    print(f"Predicted: {predictions_rescaled[i][0]:.2f}, Actual: {y_test_rescaled[i][0]:.2f}")
```

4/4 ————— 0s 73ms/step

Sample Predictions:

Predicted: 27.15, Actual: 23.60

Predicted: 38.34, Actual: 32.40

Predicted: 16.38, Actual: 13.60

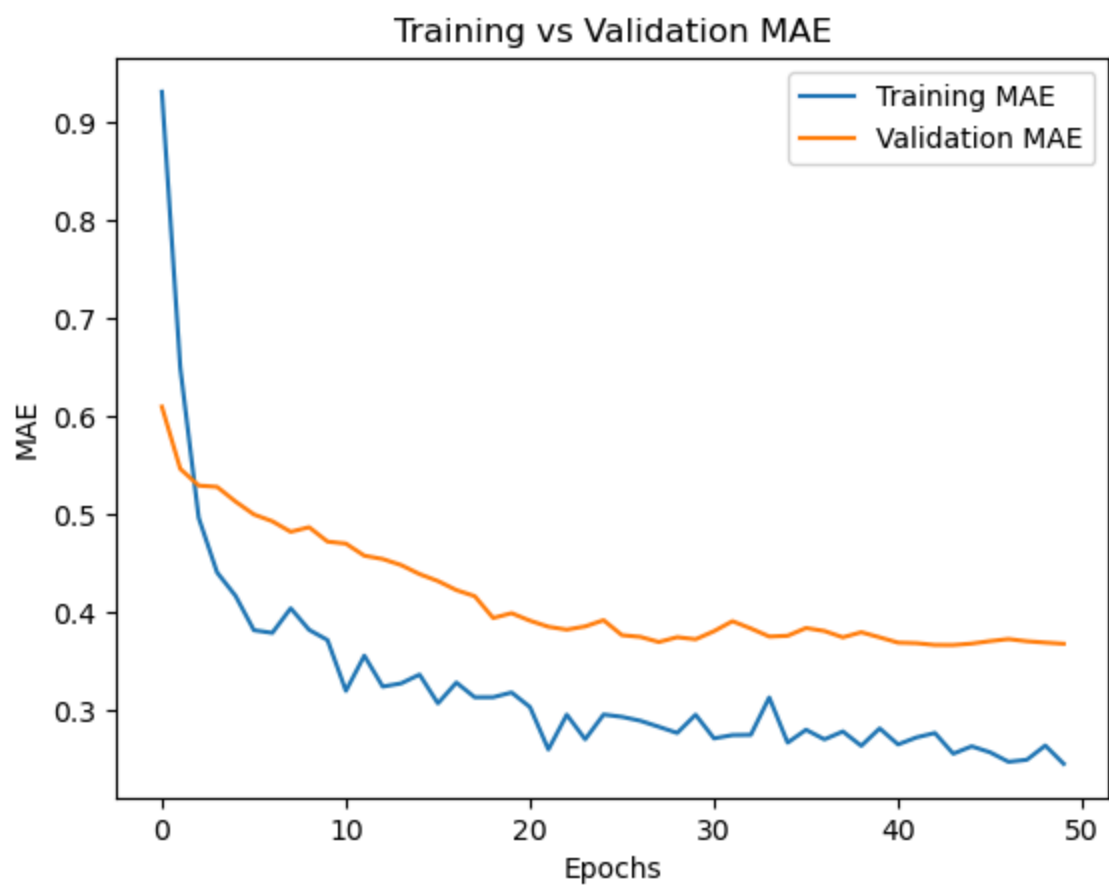
Predicted: 25.30, Actual: 22.80

Predicted: 14.85, Actual: 16.10

```
In [12]: plt.figure()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.title('Training vs Validation Loss')
plt.legend()
plt.show()
```



```
In [13]: plt.figure()
plt.plot(history.history['mae'], label='Training MAE')
plt.plot(history.history['val_mae'], label='Validation MAE')
plt.xlabel('Epochs')
plt.ylabel('MAE')
plt.title('Training vs Validation MAE')
plt.legend()
plt.show()
```

In []: